



1. 数据结构不包含的内容是
 - A. 数据的元素来源
 - B. 数据的逻辑结构
 - C. 数据的存储结构
 - D. 对数据施加的操作

2. 下列选项中，属于逻辑结构的是
 - A. 循环队列
 - B. 二叉树
 - C. 散列表
 - D. 邻接表

3. 下列选项中，属于顺序存储结构优点的是
 - A. 插入运算方便
 - B. 删除运算方便
 - C. 存储密度大
 - D. 方便存储各种逻辑结构

4. 某线性表中最常用的操作是在最后一个元素之后插入一个元素和删除第一个元素，则下列存储结构中，最节省运算时间的是
 - A. 单链表
 - B. 仅有头指针的单循环链表
 - C. 双向链表
 - D. 仅有尾指针的单循环链表

5. 用不带头结点的单链表存储队列，在进行删除运算时
 - A. 仅修改头指针
 - B. 仅修改尾指针
 - C. 头、尾指针一定都要修改
 - D. 头、尾指针可能都要修改

6. 二维数组M，行下标取值范围为0~8，列下标取值范围为1~10，若按行优先存储时，元素M[8][5]的存储地址为ar，则按列优先存储时，地址ar存储的数组元素应是
 - A. M[8][5]
 - B. M[5][8]
 - C. M[3][10]
 - D. M[0][9]

7. 根据二叉树的定义，3个结点构成的二叉树的树型有
 - A. 2种
 - B. 3种
 - C. 4种
 - D. 5种

8. 一棵有序树可转换为一棵二叉树，树的后序遍历对应二叉树的

- A.前序遍历
- B.中序遍历
- C.后序遍历
- D.以上都不对

9.若图G的邻接表中有奇数个表结点, 则G是

- A.含奇数个顶点的图
- B.无向图
- C.含偶数个顶点的图
- D.有向图

10.若用邻接矩阵存储有向图, 矩阵中主对角线以下的元素均为零, 则关于该图拓扑排序序列的结论是

- A.存在, 且唯一
- B.存在, 且不唯一
- C.存在, 可能不唯一
- D.无法确定是否存在

11.如果无向图G的最小生成树中含有边 (a, b) 和 (a, c) ,则下列选项中, 一定不在T中的边是 

- A.(b, c)
- B.(b, d)
- C.(c, d)
- D.(c, e)

12.下列排序算法中, 在每一趟都能选出一个元素放到其最终位置上的是

- A.插入排序
- B.希尔排序
- C.归并排序
- D.堆排序

13.若数据元素序列11, 13,15,7,8,9,23,2,5是采用下列排序方法之一得到的第二趟排序后的结果, 则该排序算法是

- A.冒泡排序
- B.插入排序
- C.选择排序
- D.归并排序

14.线性表采用顺序存储或链式存储, 对其进行查找的方法应是

- A.顺序查找
- B.二分查找
- C.散列查找
- D.索引查找

15.设有序表为{1,3,9,12,32,41,45,62,75,77,82},采用二分查找法查找关键字75,查找过程中关键字之间的比较次数是

- A.1
- B.2
- C.3
- D.4

02331 数据结构 2018年4月

-
- 16.在数据结构中，从逻辑上可以把数据结构分为线性结构和。
- 17.为便于实现单链表的插入及删除运算，需要在单链表中增加一个结点，该结点称为。
- 18.在二维数组A[10][8]中，每个数组元素占用4个存储单元，则数组A需要的存储单元个数是。
- 19.对长度为1的广义表A，若有Head (A) =Tail (A) ，则A=。
- 20.设高为h的二叉树T中只有度为0和2的结点，则T包含的结点数最多为。
- 21.一个连通图的是包含图中所有顶点的极小连通子图。
- 22.无向图G中含7个顶点，顶点间的边是随机设置的，为保证图G在任何情况下都是连通的，则需要的边数最少是。
- 23.求单源最短路径的迪杰斯特拉（Dijkstra）算法是按照路径 不减的次序求出各条路径的。
- 24.一组记录的关键字为（45,53,18,49,36,76,13,97,36,32）,利用快速排序方法对其进行排序，选择45为基准，一次划分后的结果为。
- 25.对箱排序的改进和推广的排序算法是。
- 26.两个栈共享数组空间data[m]（定义如下），它们的栈底分别设在数组的两端（初始化后top1=-1, top2=m）。

```
typedef struct {
    DataType data[m];
    int top1, top2;
} SeqStack;
```

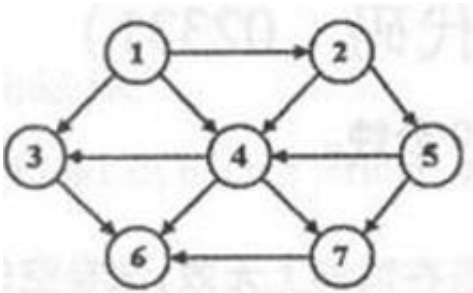
回答下列问题。

- (1) 编写判断栈满的函数 `int stackfull( SeqStack *s );`
- (2) 编写进栈函数 `void push( SeqStack *s, int si, DataType x ),`
其中，`si` 取值为 0、1，分别表示栈底为 0 或 `m-1` 的栈。

27. 已知二叉树T中含有元素 A, B, C, D, E, F, G, H, T 的前序遍历序列、中序遍历序列和后序遍历序列如下，其中符号__表示未知元素. 试写出①到⑩所代表的正确元素值.

前序遍历序列	A	B	D	①	E	F	G	②
中序遍历序列	③	B	A	④	C	G	F	⑤
后序遍历序列	⑥	⑦	⑧	H	F	C	⑨	⑩

28. 设图G如题28图所示.



题 28 图

回答下列问题。

- (1) 图G是否是有向无环图？
- (2) 给出图G所有的拓扑排序序列。

29. 设关键字序列为：53,15,72,52,48,67,63,23。已知散列表地址空间为0~11,散列函数为 $H(k) = k \bmod 11$ ，采用线性探查再散列法解决冲突。

- (1) 将所给关键字数据依次填入该散列表中；
- (2) 计算等概率下查找成功的平均查找长度。

30. 已知队列的基本操作定义如下，请在空白处填写适当的语句，完成指定的功能。

31.程序G1是将输入的m行n列的二维数组a变换为三元组表形式存储在数组b中.请在空白处填上适当内容将算法补充完整,

```
#define MAXSIZE 100
typedef int DataType;
typedef struct {
    int i, j;          // 非零元素行列下标
    DataType v;        // 非零元素值
} TriTupleNode;
typedef struct {
    TriTupleNode data[MAXSIZE]; // 存储三元组数组

    int m, n, t; // m: 矩阵的行, n: 矩阵的列, t: 非零元素数量
} TSMatrix;
void f31(TSMatrix *b, int *a, int m, int n)
// 将 m 行 n 列的矩阵 a 变换为三元组表形式存储在 b 中
{
    int i, j, k=0;
    for (i=0; i<m; i++)
        for (j=0; j<n; j++)
            {
                if ( *a !=0 )
                {
                    b->data[k].i = i;
                    b->data[k].j = j;
                    b->data[k].v = (1) ;
                    (2) ;
                }
                a++;
            }
    b->m = m;
    b->n = n;
    b->t = (3) ;
}
```

32.已知二叉树7.如题32图所示. 阅读程序f32,写出执行f32（T）的输出结果。

```

typedef char DataType;
typedef struct node
{
    DataType data;           // data 是数据域
    struct node * lchild, * rchild; // 分别指向左右孩子
} BinTreeNode;
typedef BinTreeNode * BinTree;
void f32 ( BinTree bt )
{
    if(bt==NULL)
    {
        f32 (bt->rchild);
        printf("%c", bt->data);
        f32 (bt->lchild);
    }
}

```

执行结果:



33. 阅读下列程序，写出执行结果。

```

void f33( int a[], int n )
{
    int i;
    for ( i=(n-1)/2; i>=0; i-- )
        Sift(a, i, n-1);
}

void Sift( int a[], int i, int h )
{
    int j, temp=a[i];
    j = 2*i+1;
    while ( j <= h )
    {
        if ( j<h && a[j] < a[j+1] )
            j ++;
        if ( temp >= a[j] )
            break;
        a[i] = a[j];
        i = j;
        j = 2*i+1;
    }
}

a[i] = temp;
}

```

```
int main()
{
    int i, a[10] = { 10, 20, 5, 23, 25, 62, 21, 1, 32, 9 };

    f3(a, 10);
    for (i = 0; i < 10; i++)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

执行结果：

34. 已知带有头结点的单链表定义如下：

```
typedef struct node
{
    char ch;
    struct node *next;
} ListNode;
typedef ListNode *LinkList;

请编写函数 int f34( LinkList h, char string[] ), 根据输入的字符串, 建立不含重复字符的链表。
```

答案

1.A

解析：数据结构包含的内容包括三方面内容：数据之间的逻辑关系，数据元素的存储方式，数据的运算即对数据元素施加的操作。

2.B

解析：数据的逻辑结构是对数据之间关系的描述，逻辑结构分线性结构和非线性结构，线性表、栈、对列都是线性结构，树形结构、图形结构和网状结构都是非线性结构。

3.C

解析：顺序存储时,相邻数据元素的存放地址也相邻(逻辑与物理统一);要求内存中可用存储单元的地址必须是连续的。优点：存储密度大，存储空间利用率高。缺点：插入或删除元素时不方便。

4.D

解析：有尾指针的单链表，删除最后一个元素与链表的长度无关，单循环链表有尾指针，所以时间复杂度为 $O(1)$,其他的都要 $O(n)$ 。

5.D

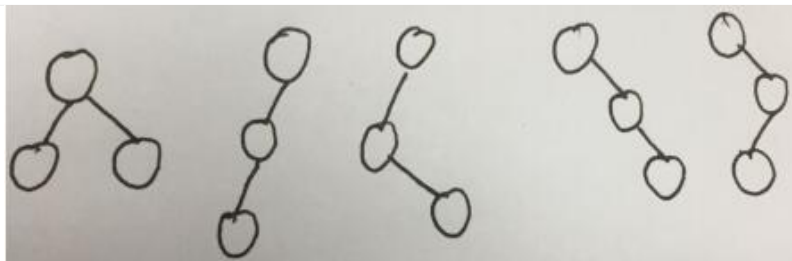
解析：因为当队列中只有一个元素时，删除此元素后要将队列置空，此时要修改队尾指针，使尾指针与头指针相等（即 $Q.rear = Q.front$ ）。假设head是指向队列的头结点指针，p为与head同类型的指针，那么head->next指向的是队列的首结点，那么出队的操作为 $p = head->next$;//指向欲出队的结点； $head->next = p->next$; $free(p)$;//销毁队首节点

6.C

解析：按行优先存储时，元素 $M[8][5]$ 的存储地址ar为第85个存储空间，则按列优先存储时，第85个存储空间的元素是 $M[3][10]$ 。

7.D

解析：3个结点能构成的二叉树如图：



8.B

解析：树转换成二叉树：首先在所有兄弟结点之间加一道连线，然后再对每个结点保留长子的连线，去掉该结点与其他孩子的连线。树的后序遍历是指先依次后序遍历根的每棵子树，然后访问根结点。二叉树的中序遍历1、中序遍历左子树 2、访问根节点 3、中序遍历右子树。

9.D

解析：无向图的邻接表的表结点的个数一定是偶数，为边数的2倍。若图G的邻接表中有奇数个表结点，则图G一定是有向图。

10.C

解析：一个有向图有拓扑序列的充要条件是该图是有向无环图。对角线以下元素均为零，表明只有顶点i到顶点j (itop1+1==s->top2;}

```
(2) 进栈Void push(SeqStack *S,int si,DataType X){if (stackfull(s)) printf( " stack overflow " );else{ if (si==0)s->data[++s->top1]=x;else s->data[--s->top2]=x;}}
```

27.

答：① C ② H ③ D ④ E ⑤ H ⑥ D ⑦ B ⑧ E ⑨ G ⑩ A

28.

答：(1) 是有向无环图 (2) 该图的拓扑排序序列有：(1, 2, 5, 4, 3, 6) 和 (1, 2, 5, 4, 3, 7, 6)

29.

答：(1) 散列表是

0	1	2	3	4	5	6	7	8	9	10	11
	67	23		15	48	72		52	53	63	

(2) $ASL_{\text{查找成功}} = 12/8 = 1.5$

30.

答：(1) $Q \rightarrow \text{rear} == Q \rightarrow \text{front}$ (2) $(Q \rightarrow \text{rear} + 1) \% \text{QueueSize}$ (3) $(Q \rightarrow \text{front} + 1) \% \text{QueueSize}$

31.

答：(1) *a ; (2) k++ ; (3) k

32.

答：7,3,9,6,1,5,2,8,4

33.

答：62,32,21,23,25,5,10,1,20,9

34.

答：int f34(LinkList h,char string[]){ LinkList p,q;Char *pc;int count=0;Pc=string;While(*pc!='\0'){P=h;While(p->next!=null){if(p->next->ch!=*pc)P=p->next;Else break;}if(p->next==null){q=(LinkList)malloc(sizeof(ListNode));q->ch=*pc;q->next=p->next;p->next=q;count++;}Pc++;}Return count;}